

Test Pratique — Architecture Microservices Spring Boot 4

Contexte

Vous allez concevoir une mini-plateforme e-commerce composée de deux microservices indépendants, d'un serveur Eureka, d'une API Gateway, et d'une application mobile.

L'ensemble est orchestré via Docker Compose.

Partie 1 — produits-service (Port 8091)

Créez un projet Spring Boot 4 (Maven, JDK 25, Spring Web, Spring Data JPA, Lombok, Spring Cache) connecté à une base **PostgreSQL** tournant dans un conteneur Docker.

Entités

Categorie : id (Long), nom (String)

Produit : id (Long), nom (String), prix (Double), stock (int), categorie (@ManyToOne → Categorie)

Données initiales (data.sql)

Insérez **3 catégories** et **5 produits** au démarrage.

Endpoints

Méthode	URL	Description
GET	/api/produits	Liste tous les produits
GET	/api/produits?categorieId={id}	Liste les produits d'une catégorie
GET	/api/produits/{id}	Détail d'un produit
POST	/api/produits	Crée un produit
GET	/api/categories	Liste toutes les catégories
GET	/api/categories/{id}	Détail d'une catégorie

Exigences techniques

- Architecture en couches : controller / service / repository / entity
- **Cache Redis** sur GET /api/produits (@Cacheable) avec invalidation sur le POST (@CacheEvict)
- **Documentation Swagger** via springdoc-openapi (accessible sur /swagger-ui.html)

Partie 2 — avis-service (Port 8092)

Créez un second projet Spring Boot 4 (même stack, sans Redis) avec sa propre base PostgreSQL conténérisée.

Entité

Avis : `id` (Long), `produitId` (Long), `auteur` (String), `commentaire` (String), `note` (int, 1–5)

Endpoints

Méthode	URL	Description
GET	<code>/api/avis/{produitId}</code>	Liste les avis d'un produit
POST	<code>/api/avis</code>	Soumet un avis

Exigences techniques

- Même architecture en couches que `produits-service`
- Documentation Swagger
- Gestion des erreurs HTTP (404 si produit inconnu, **via appel Feign** — voir Partie 3)

Partie 3 — Infrastructure (Eureka · Feign · API Gateway)

Eureka Server

Créez un projet `eureka-server` (port **8761**) annoté `@EnableEurekaServer`. Les deux microservices s'y enregistrent avec leur `spring.application.name`.

Feign dans avis-service

Dans `avis-service`, déclarez un `@FeignClient(name = "produits-service")` pour vérifier l'existence du produit (`GET /api/produits/{id}`) avant d'enregistrer un avis. Retournez `404` si le produit est introuvable.

API Gateway (Port 8090)

Créez un projet `api-gateway` (Spring Cloud Gateway) qui route :

- `/api/produits/**` → `produits-service`
- `/api/categories/**` → `produits-service`
- `/api/avis/**` → `avis-service`

Partie 4 — Docker Compose

Fournissez un `Dockerfile` pour chaque microservice et un `docker-compose.yml` unifié à la racine du projet.

Service	Port	Dépend de
postgres	5432	—
redis	6379	—
eureka-server	8761	—
produits-service	8091	postgres · redis · eureka-server
avis-service	8092	postgres · eureka-server
api-gateway	8090	eureka-server

Partie 5 — Application Mobile (Flutter ou React Native)

Développez une application mobile qui :

1. Récupère la liste des catégories via l'API Gateway (`GET /api/categories`)
2. Permet à l'utilisateur de sélectionner une catégorie et affiche les produits correspondants (`GET /api/produits?categorieId={id}`)
3. Au clic sur un produit, affiche ses avis (`GET /api/avis/{produitId}`)

Toutes les requêtes passent **exclusivement par l'API Gateway** (`http://<IP>:8090`).

Flutter : utilisez `http`, `DropDownButton`, `FutureBuilder`, `ListView.builder`.

React Native : utilisez `fetch` ou `axios`, `Picker` (ou `FlatList`), `useEffect`.

Partie 6 — Git & Tests

Dépôt GitHub

Créez un dépôt unique avec la structure suivante :

```
/projet-boutique/  
├── produits-service/  
├── avis-service/  
├── eureka-server/  
├── api-gateway/  
├── mobile-app/  
├── docker-compose.yml  
└── README.md
```

Créez deux branches : `version1` (Parties 1–4) et `version2` (Parties 5–6).

Tests (branche `version2`)

Ajoutez dans `produits-service` :

- **Test unitaire** : tester la logique du service (`ProduitService`) avec Mockito
 - **Test d'intégration** : tester le repository avec `@DataJpaTest` et une base H2 ou Testcontainers
 - **Test E2E** : avec **Cypress**, simuler le parcours `liste produits → détail produit → avis` via l'API Gateway directement via `cy.request()`
-

Livraison : lien GitHub unique contenant tout le code et toutes les versions !, avec un `README.md` expliquant comment lancer le projet et comment exécuter les tests.